디지털 시스템 설계 — 수업정리노트

Ch0: Introduction & Trend in Computer Systems

과목 개요

이 과목 “Digital Systems Design”은 서울대 심재웅 교수님이 삼성DS System Expert 과정을 위해 진행하는 수업이다. 핵심 목표는 CPU/GPU/TPU와 같은 작은 하드웨어 조각을 직접 설계할 수 있는 능력을 기르는 것이다.

왜 이 과목이 필요한가? 소프트웨어 엔지니어가 하드웨어의 동작 원리를 이해하면, 시스템 레벨에서의 최적화 판단이 가능해진다. 특히 삼성DS처럼 반도체를 설계하는 조직에서는 SW/HW 인터페이스를 이해하는 것이 핵심 역량이다.

과목에서 다루는 주제는 시스템 스택의 Logic 레벨에 해당한다:

```
Algorithms
  └ Prog. Model/Lang.
       └ System Software
            └ Compiler
                 └ SW/HW Interface
                      └ Micro-architecture
                           └ Logic          ◀◀◀ 여기!
                                └ Devices
                                     └ Physics
```

구체적 토픽: 조합/순차 논리 복습 → Verilog HDL → 수 체계 & 부동소수점 → 하드웨어 빌딩 블록(Adder, Multiplier, SRAM, DRAM) → DNN 소개 → FPGA로 CNN 가속기 구현

컴퓨터 시스템의 트렌드

1971년 Intel 4004(2,300 트랜지스터, 0.74MHz)에서 시작하여 오늘날 540억 개 이상의 트랜지스터를 가진 Manycore 프로세서까지, 2천만 배 이상 증가했다.

```
1971: 4004 (0.74MHz) ← Clock Speed 향상
1978: 8086 (10MHz)   ← Bus Width 확장
1993: Pentium (200MHz, Superscalar) ← Pipeline Width 확장
2006: Core2Duo (3.3GHz, Dual Core) ← Core 수 증가
TODAY: Manycore
```

Moore’s Law: 칩 위의 트랜지스터 수가 약 12~24개월마다 2배로 증가. 50년간 잘 맞았으나 현재 둔화.

Dennard Scaling: $P \propto C \cdot V^2 \cdot F$. 트랜지스터 축소 → C↓, V↓ → F↑해도 전력 유지. 하지만 2000년대 중반 이후 V를 더 낮출 수 없게 되어 종료. 이것이 주파수 경쟁 → 멀티코어 시대 전환의 이유.

성능 향상 원천(AMD Lisa Su): 공정기술 40%, 마이크로아키텍처 17%, 전력관리 15%, 다이크기 12%, TDP 8%, 컴파일러 8%.

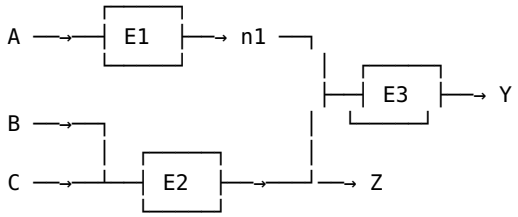
특수 가속기: Google TPU v1(28nm, 331mm², 92 TOPs INT8, 40W), TPU v2/v3(HBM, bfloat16, 105 TFLOPs), Tesla FSD(Samsung 14nm, 260mm², 36W, NPU 2개), AWS Inferentia(128 TOPS), Intel Loihi(뉴로모픽, 14nm 60mm²).

Ch1: Review — Logic Design

1. Logic Gates

Gate	Boolean Eq.	Truth Table (A,B→Y)
BUF	$Y = A$	0→0, 1→1
NOT	$Y = \overline{A}$	0→1, 1→0
AND	$Y = A \cdot B$	00→0,01→0,10→0,11→1
OR	$Y = A+B$	00→0,01→1,10→1,11→1
XOR	$Y = A \oplus B$	00→0,01→1,10→1,11→0
NAND	$Y = \overline{(A \cdot B)}$	00→1,01→1,10→1,11→0
NOR	$Y = \overline{(A+B)}$	00→1,01→0,10→0,11→0

2. Digital Circuits



조합(Combinational): 출력 = f(현재 입력만), 메모리 없음. 순차(Sequential): 출력 = f(현재+과거 입력), 메모리 있음.

3. Boolean Equations

Minterm/Maxterm 표(3변수):

A	B	C	Minterm	Name	Maxterm	Name
0	0	0	$\overline{A} \cdot \overline{B} \cdot \overline{C}$	m0	$A+B+C$	M0
0	0	1	$\overline{A} \cdot \overline{B} \cdot C$	m1	$A+B+\overline{C}$	M1
0	1	0	$\overline{A} \cdot B \cdot \overline{C}$	m2	$A+\overline{B}+C$	M2
0	1	1	$\overline{A} \cdot B \cdot C$	m3	$A+\overline{B}+\overline{C}$	M3
1	0	0	$A \cdot \overline{B} \cdot \overline{C}$	m4	$\overline{A}+B+C$	M4
1	0	1	$A \cdot \overline{B} \cdot C$	m5	$\overline{A}+B+\overline{C}$	M5
1	1	0	$A \cdot B \cdot \overline{C}$	m6	$\overline{A}+\overline{B}+C$	M6
1	1	1	$A \cdot B \cdot C$	m7	$\overline{A}+\overline{B}+\overline{C}$	M7

SOP: Y=1인 행의 minterm OR. POS: Y=0인 행의 maxterm AND.

4. Boolean Algebra

Axioms: A1-A5(Binary, NOT, AND/OR)와 각 dual.

단일변수 정리: T1(Identity: $B \cdot 1 = B$), T2(Null: $B \cdot 0 = 0$), T3(Idempotency: $B \cdot B = B$), T4(Involution: $\bar{\bar{B}} = B$), T5(Complements: $B \cdot \bar{B} = 0$).

다변수 정리: - T6(Commutativity), T7(Associativity), T8(Distributivity) - T9(Covering: $B \cdot (B+C) = B$), T10(Combining: $BC + B\bar{C} = B$ ★핵심!) - T11(Consensus: $BC + \bar{B}D + CD = BC + \bar{B}D$) - T12(De Morgan's: $\overline{(B_0 \cdot B_1 \ldots)} = \bar{B}_0 + \bar{B}_1 \ldots$ ★핵심!)

De Morgan → NAND/NOR만으로 모든 회로 구현 가능(Logical Completeness).

간소화 예제: $\bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C = \bar{A}C(\bar{B}+B) + A\bar{B}C = \bar{A}C + A\bar{B}C = C(\bar{A}+\bar{B})$.

5. K-Map

Gray Code 순서(00,01,11,10). 규칙: 1 모두 커버, 1만 포함, 2의 거듭제곱 크기, 최대한 크게, wrap-around, 중복 가능.

6. Combinational Building Blocks

Mux: $Y = D_0 \cdot \bar{S} + D_1 \cdot S$. 2^(N-1) 입력 Mux로 N 입력 함수 구현(LUT, FPGA 원리).

```
D₀ ──┤ 0 │
     │   ├── Y        Y = D₀·S̄ + D₁·S
D₁ ──┤ 1 │
     └───┘
       S
```

Decoder: N입력 2^N 출력, 하나만 assert. OR과 결합하면 모든 함수 구현.

X(unknown), Z(high-impedance): 실제 회로의 특수 상태.

7. Sequential Circuits

Bistable element: 인버터 2개 루프, 상태 변경 불가 → Latch/FF 필요.

```
┌─────┐      ┌─────┐
│ I1  ├──→───┤ I2  ├──→──┐
└─────┘      └─────┘     │
    ↑                    │
    └────────────────────┘ Q
```

SR Latch: S=R=0→유지, S=1→Set, R=1→Reset, S=R=1→금지.

```
R──→┤NOR├──→ Q          S  R │ Q      Q̄
    │ N1│                0  0 │ Qprev  Q̄prev
    └───┘                0  1 │ 0      1
S──→┤NOR├──→ Q̄          1  0 │ 1      0
    │ N2│                1  1 │ 금지!
    └───┘
```

D Latch: CLK=1 transparent, CLK=0 opaque.

D Flip-Flop: master+slave D Latch. Rising edge에서만 D→Q.

```
       CLK           CLK
D ──→┤D Q├─N1──→┤D Q├──→ Q
     │L1 Q̄├     │L2 Q̄├──→ Q̄
     └────┘     └────┘
```

```
master      slave
```

8. FSM

FSM = next state logic + register + output logic. Moore: 출력 =f(state), Mealy: 출력=f(state+input).

```
Moore: inputs → [NSL] → [REG] → [OL] → outputs
                                ↑ state만
Mealy: inputs → [NSL] → [REG] → [OL] → outputs
                                ↑ state+inputs
```

Traffic Light 예제: $S'_1=S_1 \oplus S_0$, $S'_0=\bar{S}_1\bar{S}_0\bar{T}_A+S_1\bar{S}_0\bar{T}_B$. Output: $L_A1=S_1$, $L_A0=\bar{S}_1S_0$, $L_B1=\bar{S}_1$, $L_B0=S_1S_0$ → Moore Machine.

Ch2: Introduction to Verilog

1. HDL의 핵심 직관

SW: 명령문을 순서대로 실행. HDL: 회로를 텍스트로 기술, 모든 것이 동시에 (concurrently) 동작. assign은 "연결"이지 "복사"가 아니다.

합성(Synthesis): HDL → 게이트/와이어(netlist). initial, #delay, $display는 합성 불가(시뮬레이션 전용).

2. Module

```
  ┌──────────────────────────┐
  │          Module          │
  │  ┌────────────────────┐  │
a─┤→│   Core Circuit     │→─┤─→ y
b─┤→│   (Internal/Body)  │  │
  │  └────────────────────┘  │
  └──────────────────────────┘
```

```
module test (input a, input b, output y);
  // 회로 기술
endmodule
```

3. 숫자 표현

형식: `<size>'<base>value`. size 생략→32비트, base 생략→10진.

```
8'hAA  → 10101010     6'hCA  → 001010 (truncation)
16'bZ  → zzzzzzzzzzzzzzzz
-14'h1234 → ffffedcc (2의 보수)
```

4. wire vs reg

wire: assign의 LHS, 값 저장 불가. reg: always/initial의 LHS, 반드시 물리 레지스터는 아님!

5. 세 가지 모델링 레벨 — 모든 코드와 회로도

Structural: Half Adder (gate-level)

```
module half_adder(input x, y, output s, c);
  xor (s, x, y);
  and (c, x, y);
endmodule
```

합성 회로도:

```
x ──→┤ XOR ├──→ s     (S = X ⊕ Y)
y ──→┤      │
     └──────┘

x ──→┤ AND ├──→ c     (C = X · Y)
y ──→┤      │
     └──────┘
```

Structural: Full Adder (모듈 인스턴스화)

```
module full_adder(input x, y, c_in, output sum, c_out);
  wire s1, c1, c2;
  half_adder ha_1(x, y, s1, c1);
  half_adder ha_2(c_in, s1, sum, c2);
  or (c_out, c1, c2);
endmodule
```

합성 회로도:

```
       ha_1                ha_2
x ──→┤XOR├──→s1──→┤XOR├──→ sum
y ──→┤   │        │   │
     └───┘   c_in──→┤   │
x ──→┤AND├──→c1     └───┘
y ──→┤   │   c_in──→┤AND├──→c2
     └───┘          │   │
                    └───┘

       c1──→┤ OR ├──→ c_out
       c2──→┤    │
            └────┘
```

Dataflow: Full Adder

```
module full_adder_dataflow(input x, y, c_in,
                           output sum, c_out);
  assign {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도:

```
x[0] ──→┐
y[0] ──→┤ + ├──→ sum[0] (= sum)
c_in ──→┤   ├──→ sum[1] (= c_out)
        └───┘

(1-bit full adder, {c_out,sum} = 2비트 결과)
```

Behavioral: Full Adder

```
module full_adder_behavioral(input x, y, c_in,
                             output reg sum, c_out);
  always @(x, y, c_in)
    {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도: Dataflow와 동일 (조합 논리, 메모리 소자 없음).

Mixed-Style: Full Adder

```
module full_adder_mixed(input x, y, c_in,
                        output sum, output reg c_out);
  wire s1, c1, c2;
  // Structural: HA1
  xor xor_ha1(s1, x, y);
  and and_ha1(c1, x, y);
  // Dataflow: HA2
  assign sum = c_in ^ s1;
  assign c2  = c_in & s1;
  // Behavioral: OR
  always @(c1, c2) c_out = c1 | c2;
endmodule
```

합성 회로도:

```
        HA1(Structural)     HA2(Dataflow)    OR(Behavioral)
x ──→┤XOR├─→s1──→┤XOR├──→ sum
y ──→│   │       │   │
     └───┘   c_in─→│   │
x ──→┤AND├─→c1    └───┘
y ──→│   │   c_in─→┤AND├─→c2
     └───┘         │   │
                   └───┘
       c1──→│ OR ├──→ c_out
       c2──→│    │
            └────┘
```

6. Port Connection

```
half_adder ha_1 (x, y, s1, c1);              // positional (비추천)
half_adder ha_2 (.x(cin), .y(s1), .s(s), .c(c2)); // named (추천!)
```

7. Simulation

```
`timescale 1ns / 100ps
module tb;
  reg clk;
  always #5 clk = ~clk;    // 주기 10ns 클럭
  initial begin
    $monitor("Time=%0t clk=%0d", $time, clk);
    #5 clk = 0; #80 $finish;
  end
endmodule
```

클럭 파형:

```
clk: _____|_____|_____|_____|_____|_____|_____
     0    5    10   15   20   25   30   35
```

Ch3: Dataflow Modeling

1. Continuous Assignment 규칙

assign은 net(wire)에만 값을 구동. Always Active. LHS는 절대 reg 아님.

```
wire out;
assign out = in1 & in2;     // regular (추천)
wire out = in1 & in2;       // implicit (OK)
```

2. Delays

Inertial delay(assign #10): 짧은 펄스 무시. Transport delay(wire #10): 모든 변화 전파.

3. Signed/Unsigned 상수 (시험 핵심)

’s’는 비트패턴 안 바꿈, 해석만 바꿈:

```
4'sb1001 → 1001 → signed → -7      5'sb1001 → 01001 → signed → +9
4'shf    → 1111 → signed → -1      4'hf     → 1111 → unsigned → 15
4'sd12   → 1100 → signed → -4      5'sd12   → 01100 → signed → +12
```

Signed/Unsigned 비교 함정: 한쪽이라도 unsigned이면 모두 unsigned로 비교!

```
wire signed [3:0] a = -4'd4;   // 1100 (signed -4)
wire unsigned [3:0] b = 4'd5;  // 0101
(a > b) → 1  // 1100=12 > 0101=5 → true (unsigned 비교!)
```

4. Operators와 회로도

4-bit Adder

```
module four_bit_adder(input [3:0] x, y, input c_in,
                      output [3:0] sum, output c_out);
  assign {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도:

```
x[3:0] ──→┐
y[3:0] ──→│  4-bit  ├──→ sum[3:0]
c_in   ──→│  Adder  ├──→ c_out
          └─────────┘

내부: {c_out, sum} = 5비트 결과
```

4-bit Adder/Subtractor — 핵심 트릭

```
module twos_adder(input [3:0] x, y, input c_in,
                  output [3:0] sum, output c_out);
  wire [3:0] t;
  assign t = y ^ {4{c_in}};
  assign {c_out, sum} = x + t + c_in;
endmodule
```

합성 회로도:

```
c_in=0: t=y (덧셈)     c_in=1: t=~y (뺄셈, x+~y+1=x-y)

            {4{c_in}}
c_in ──→┤Replicate├──→ 4'b0000 or 4'b1111
          └──────────┘
```

```
y[3:0] ──→┤  XOR     ├──→ t[3:0]
{4{c_in}}→┤  (4-bit)│
          └─────────┘

x[3:0] ──→┐
t[3:0] ──→┤  4-bit   ├──→ sum[3:0]
c_in   ──→┤  Adder   ├──→ c_out
          └─────────┘
```

Parity Generator (Reduction XOR)

```
module parity_gen(input [8:0] x, output ep, op);
  assign ep = ^x;     // even parity
  assign op = ~ep;    // odd parity
endmodule
```

합성 회로도:

```
x[0]──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├──→ ep ──→┤NOT├──→ op
x[1]──→┤   │   │   │   │   │   │   │          └───┘
       └───┘   │   │   │   │   │   │
x[2]──→┤XOR├──→┤   │   │   │   │   │
x[3]──→┤   │   └───┘   │   │   │   │
       └───┘           │   │   │   │
x[4]──→┤XOR├──→┤XOR├──→┤   │   │   │
x[5]──→┤   │   │   │   └───┘   │   │
       └───┘   │   │           │   │
x[6]──→┤XOR├──→┤   │           │   │
x[7]──→┤   │   └───┘           │   │
       └───┘                   │   │
x[8]──────────────────────────→┤   │
                               └───┘
```

All-Zero / All-One Detector

```
module all_bit_detector(input [7:0] x, output zero, one);
  assign zero = ~(|x);   // OR reduction → NOT
  assign one  = &x;      // AND reduction
endmodule
```

합성 회로도:

```
x[0]┐                          x[0]┐
x[1]┤                          x[1]┤
x[2]┤ 8-input                  x[2]┤ 8-input
x[3]┤   OR   ──→┤NOT├──→ zero  x[3]┤  AND  ──→ one
x[4]┤  gate  │  └───┘          x[4]┤  gate │
x[5]┤        │                 x[5]┤       │
x[6]┤        │                 x[6]┤       │
x[7]┘        │                 x[7]┘       │
             └┘                            └┘
```

4:1 Mux (Conditional)

```
module mux4_to_1(input i0, i1, i2, i3, input s1, s0, output out);
  assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);
endmodule
```

합성 회로도:

```
i0 ──→┤ 0 │              s0
```

```
i1 ──→┤ 1 ├──→┤ 0 │       │
      └───┘    │    ├──→ out
   s0          │    │
i2 ──→┤ 0 │  ┤ 1 │
i3 ──→┤ 1 ├──→┤    │
      └───┘    └────┘

   s0           s1
```

5. Shift Operators

```
8비트 signed 10101010 (-86):
Logical  >>1: 01010101       (0 패딩)
Arith   >>>1: 11010101 (-43) (부호비트 1로 패딩)
Logical  <<1: 01010100       (0 패딩)
Arith   <<<1: 01010100 (84)  (0 패딩, 부호 변경!)
```

6. Bitwise x/z 진리표

&	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

|	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

7. Vectors, Arrays, Memory

```
reg [7:0] a = 14;  // a[3:0]=1110 ✓
reg [0:7] b = 14;  // b[0:3]=0000 ✓, b[3:0] 불법!

wire [15:0] v;
v[8+:8]  // = v[15:8]    v[7-:8]  // = v[7:0]
```

Array: 한 번에 하나의 word만 접근. mema[1]=0 합법, mema=0 불법.

Ch4: Behavioral Modeling

1. initial vs always

initial: 시뮬레이션 중 1회 실행, ASIC 합성 불가(테스트벤치에서만). always: sensitivity list 이벤트마다 실행, 합성 가능.

```
reg clk = 0;                          // combined 초기화
module inc(input [3:0] x,
           output reg [3:0] y = 4'b1000); // 포트+초기화 결합
endmodule
```

2. always Block — 순차/조합 논리와 회로도

D Flip-Flop (순차 논리)

```
module flop(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(posedge CLK)
    Q <= D;    // nonblocking
endmodule
```

합성 회로도 (4-bit register = 4개 DFF):

```
D[0]──→┤D  Q├──→Q[0]
       │DFF │
CLK──→┤>   │
       └────┘
D[1]──→┤D  Q├──→Q[1]
       │DFF │
CLK──→┤>   │
       └────┘
D[2]──→┤D  Q├──→Q[2]
       │DFF │
CLK──→┤>   │
       └────┘
D[3]──→┤D  Q├──→Q[3]
       │DFF │
CLK──→┤>   │
       └────┘
```

D Latch (순차 논리)

```
module my_latch(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(CLK, D)
    if (CLK == 1) Q <= D;
endmodule
```

합성 회로도 (4-bit latch):

```
D[0]──→┤D  Q├──→Q[0]     CLK=1: transparent (Q=D)
       │LAT │             CLK=0: opaque (Q 유지)
CLK──→┤E   │
       └────┘
(×4 for each bit)
```

Inverter (조합 논리 — reg이지만 FF 아님!)

```
module inverter(input [3:0] a, output reg [3:0] y);
  always @(*)
    y = ~a;    // blocking
endmodule
```

합성 회로도 (메모리 소자 없음!):

```
a[0]──→┤NOT├──→y[0]
       └───┘
a[1]──→┤NOT├──→y[1]
       └───┘
a[2]──→┤NOT├──→y[2]
       └───┘
a[3]──→┤NOT├──→y[3]
       └───┘
★ reg 타입이지만 합성 시 FF 없음! reg ≠ 물리 레지스터!
```

4:2 Encoder (조합 + unwanted latch 방지)

```
module encoder4to2(input Y0, Y1, Y2, Y3,
                   output reg [1:0] A);
  always @* begin
    if      (Y0 && !Y1 && !Y2 && !Y3) A = 2'b00;
    else if (!Y0 && Y1 && !Y2 && !Y3) A = 2'b01;
```

```
    else if (!Y0 && !Y1 && Y2 && !Y3) A = 2'b10;
    else if (!Y0 && !Y1 && !Y2 && Y3) A = 2'b11;
    else A = 2'b00;  // ★ else 필수! 없으면 latch 합성!
  end
endmodule
```

합성 회로도 (else 있을 때: 순수 조합 논리):

```
Y0─┐                        ┌─→ A[0]
Y1─┤ Priority Encoder ──┤
Y2─┤ (조합 논리)          └─→ A[1]
Y3─┘
★ else 없으면 → Latch가 합성됨 (의도치 않은 메모리!)
```

4:1 Mux using if...else

```
module mux4_to_1_ifelse(input i0, i1, i2, i3,
                        input s1, s0, output reg out);
  always @(*)
    if (s1) begin
      if (s0) out = i3; else out = i2;
    end else begin
      if (s0) out = i1; else out = i0;
    end
endmodule
```

합성 회로도:

```
i0──→┤0│     s0 선택
i1──→┤1├──→┤0│     s1 선택
     └─┘    │ ├──→ out
i2──→┤0│    │ │
i3──→┤1├──→┤1│
     └─┘    └─┘
  s0         s1
```

3. Blocking vs Nonblocking — 핵심!

실행 순서 차이

```
// Blocking: 순서대로, 즉시 할당
initial begin
  x = #5 1'b0;   // time 5
  y = #3 1'b1;   // time 8 (5+3, blocked)
  z = #6 1'b0;   // time 14 (8+6)
end

// Nonblocking: RHS 동시 평가
initial begin
  x <= #5 1'b0;  // time 5  (모두 time 0에서 동시 시작!)
  y <= #3 1'b1;  // time 3
  z <= #6 1'b0;  // time 6
end
```

Shift Register — 결정적 차이

```
// Blocking (잘못! FF 1개만 합성)
always @(posedge clk) begin
  qout[0] = sin;       // 즉시 변경
```

```
    qout[1] = qout[0];   // sin의 새 값! (=sin)
    qout[2] = qout[1];   // 역시 sin!
    qout[3] = qout[2];   // 모두 sin!
  end
```

합성 결과 (잘못됨):

```
sin ──→┤DFF├──→ qout[0]=qout[1]=qout[2]=qout[3]
       └───┘

clk     (FF 1개만!)
```

```
  // Nonblocking (올바름! FF 4개 체인)
  always @(posedge clk) begin
    qout[0] <= sin;       // 모든 RHS는 이전 값
    qout[1] <= qout[0];   // qout[0]_old
    qout[2] <= qout[1];   // qout[1]_old
    qout[3] <= qout[2];   // qout[2]_old
  end
  // 더 좋은 표현: qout <= {qout[2:0], sin};
```

합성 결과 (올바름):

```
sin──→┤DFF├──→qout[0]──→┤DFF├──→qout[1]──→┤DFF├──→qout[2]──→┤DFF├──→(
      └───┘             └───┘             └───┘             └───┘
                                clk (공유)
```

Race Condition 방지

```
  // Blocking → race! (순서 의존)      // Nonblocking → 올바른 swap
  always @(posedge clock) x = y;       always @(posedge clock) x <= y;
  always @(posedge clock) y = x;       always @(posedge clock) y <= x;
```

Nonblocking 3단계: (1) 모든 RHS 읽기 (2) 수식 평가→임시 저장 (3) 임시 →LHS 할당.

count/finish 예제

count=1, finish=0에서 posedge clk:

```
  // Blocking → finish=1              // Nonblocking → finish=0
  always @(posedge clk) begin          always @(posedge clk) begin
    count = count-1;    // count=0       count <= count-1;    // RHS:
1-1=0
    if(count==0)                         if(count==0)         // count
는 아직 1!
      finish = 1;       // true!           finish <= 1;       //
false!
  end                                  end
```

4. 5가지 가이드라인 (필수 암기!)

```
1. 순차 논리: always @(posedge clk) + nonblocking (<=)
2. 간단한 조합 논리: assign (dataflow)
3. 복잡한 조합 논리: always @(*) + blocking (=)
4. 같은 신호를 여러 always/assign에서 할당하지 마라
5. blocking과 nonblocking을 같은 블록에서 섞지 마라
```

5. Counter with 비동기 리셋

```
module counter(input clock, clear, output reg [3:0] qout);
  always @(negedge clock or posedge clear) begin
    if (clear) qout <= 4'd0;       // 비동기 리셋 우선
    else       qout <= qout + 1;   // negedge에서 카운트
  end
endmodule
```

합성 회로도:

```
qout[3:0]──→┤ +1 ├──→┤ 0 │
            └────┘   │MUX├──→┤D    Q├──→ qout[3:0]
4'd0 ──────────────→┤ 1 │   │DFF  │
                     └───┘   │  R  │
                clear──→ sel  └────┘
                              negedge clock
                (clear=1이면 비동기 즉시 리셋)
```

6. case/casex/casez

```
module mux_4x1_case(input I0,I1,I2,I3, input [1:0] S, output reg Y);
  always @(*)
    case (S)
      2'b00: Y=I0; 2'b01: Y=I1; 2'b10: Y=I2; 2'b11: Y=I3;
    endcase
endmodule
```

합성 회로도 (4:1 Mux):

```
I0──→┤       ├
I1──→┤  4:1  ├──→ Y
I2──→┤  MUX  │
I3──→┤       │
     └───────┘
S[1:0]──→┤sel│
```

```
// Trailing zero counter (casex)
module trailing_zero_4b(input [3:0] data, output reg [2:0] out);
  always @(data)
    casex (data)
      4'bxxx1: out=0;  4'bxx10: out=1;
      4'bx100: out=2;  4'b1000: out=3;
      4'b0000: out=4;  default: out=3'b111;
    endcase
endmodule
```

합성 회로도 (priority encoder):

```
data[0]──→┐
data[1]──→┤ Priority   ├──→ out[2:0]
data[2]──→┤ Encoder    │
data[3]──→┤ (casex에서 │
          │  x=don't   │
          │  care)     │
          └────────────┘
```

7. Timing Control

Inter-assignment: #10 a=b; → 10단위 후 문장 전체 실행. Intra-assignment: a=#10 b; → RHS 즉시 평가, 10단위 후 할당.

posedge: 0→1,0→x,0→z,x→1,z→1. negedge: 1→0,1→x,1→z,x→0,z→0.

Named event: event declared → trigger(->) → recognized(@). Level-sensitive: wait(condition).

8. Loop Constructs

Zero counter (for)

```
module zero_count_for(input [7:0] data, output reg [3:0] out);
  integer i;
  always @(data) begin
    out = 0;
    for (i=0; i<=7; i=i+1)
      if (data[i]==0) out = out + 1;
  end
endmodule
```

합성 회로도 (for는 unroll됨):

```
data[0]──→┤==0?├──→┐
data[1]──→┤==0?├──→┤
data[2]──→┤==0?├──→┤ Population
data[3]──→┤==0?├──→┤ Count    ──→ out[3:0]
data[4]──→┤==0?├──→┤ (zero의
data[5]──→┤==0?├──→┤  개수)
data[6]──→┤==0?├──→┤
data[7]──→┤==0?├──→┘
```

8-bit Left Shift Register (for + nonblocking)

```
module lshift_reg(input clk, rstn, input [7:0] load_val,
                  input load_en, output reg [7:0] op);
  integer i;
  always @(posedge clk) begin
    if (!rstn) op <= 0;
    else if (load_en) op <= load_val;
    else begin
      for (i=0; i<7; i=i+1)
        op[i+1] <= op[i];   // 각 비트가 다음으로 시프트
      op[0] <= op[7];       // circular
    end
  end
endmodule
```

합성 회로도:

```
op[7]──→op[0]──→op[1]──→op[2]──→...──→op[7]
       (circular left shift, 8개 DFF 체인)

rstn=0: 전체 0으로 리셋
load_en=1: load_val 로드
else: 매 posedge마다 좌로 1비트 순환 시프트
```

repeat: 고정 횟수 반복. forever: 무한 반복, $finish/disable로 종료.

```
// forever로 비대칭 클럭 생성
```

```
initial begin
  clock <= 0;
  forever begin #10 clock<=1; #5 clock<=0; end
end
```

```
clock: _____|__________|_____|__________|_____
        0    5         15    20         30
        (HIGH=10ns, LOW=5ns, 주기=15ns)
```

Ch4: Behavioral Modeling

1. initial vs always

initial: 시뮬레이션 중 1회 실행, ASIC 합성 불가(테스트벤치에서만). always: sensitivity list 이벤트마다 실행, 합성 가능.

```
reg clk = 0;                          // combined 초기화
module inc(input [3:0] x,
           output reg [3:0] y = 4'b1000); // 포트+초기화 결합
endmodule
```

2. always Block — 순차/조합 논리와 회로도

D Flip-Flop (순차 논리)

```
module flop(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(posedge CLK)
    Q <= D;    // nonblocking
endmodule
```

합성 회로도 (4-bit register = 4개 DFF):

```
D[0]──→┤D  Q├──→Q[0]
       │DFF │
CLK──→┤>   │
       └────┘
D[1]──→┤D  Q├──→Q[1]
       │DFF │
CLK──→┤>   │
       └────┘
D[2]──→┤D  Q├──→Q[2]
       │DFF │
CLK──→┤>   │
       └────┘
D[3]──→┤D  Q├──→Q[3]
       │DFF │
CLK──→┤>   │
       └────┘
```

D Latch (순차 논리)

```
module my_latch(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(CLK, D)
    if (CLK == 1) Q <= D;
endmodule
```

합성 회로도 (4-bit latch):

```
D[0]──→┤D  Q├──→Q[0]     CLK=1: transparent (Q=D)
       │LAT │             CLK=0: opaque (Q 유지)
CLK──→┤E   │
       └────┘
(×4 for each bit)
```

Inverter (조합 논리 — reg이지만 FF 아님!)

```
module inverter(input [3:0] a, output reg [3:0] y);
  always @(*)
    y = ~a;    // blocking
endmodule
```

합성 회로도 (메모리 소자 없음!):

```
a[0]──→┤NOT├──→y[0]
       └───┘
a[1]──→┤NOT├──→y[1]
       └───┘
a[2]──→┤NOT├──→y[2]
       └───┘
a[3]──→┤NOT├──→y[3]
       └───┘
★ reg 타입이지만 합성 시 FF 없음! reg ≠ 물리 레지스터!
```

4:2 Encoder (조합 + unwanted latch 방지)

```
module encoder4to2(input Y0, Y1, Y2, Y3,
                   output reg [1:0] A);
  always @* begin
    if      (Y0 && !Y1 && !Y2 && !Y3) A = 2'b00;
    else if (!Y0 && Y1 && !Y2 && !Y3) A = 2'b01;
    else if (!Y0 && !Y1 && Y2 && !Y3) A = 2'b10;
    else if (!Y0 && !Y1 && !Y2 && Y3) A = 2'b11;
    else A = 2'b00;  // ★ else 필수! 없으면 latch 합성!
  end
endmodule
```

합성 회로도 (else 있을 때: 순수 조합 논리):

```
Y0─┐                    ┌─→ A[0]
Y1─┤ Priority Encoder ──┤
Y2─┤ (조합 논리)        └─→ A[1]
Y3─┘
★ else 없으면 → Latch가 합성됨 (의도치 않은 메모리!)
```

4:1 Mux using if...else

```
module mux4_to_1_ifelse(input i0, i1, i2, i3,
                        input s1, s0, output reg out);
  always @(*)
    if (s1) begin
      if (s0) out = i3; else out = i2;
    end else begin
      if (s0) out = i1; else out = i0;
    end
endmodule
```

합성 회로도:

```
i0──→┤0│      s0 선택
i1──→┤1├──→┤0│      s1 선택
     └─┘    │ ├──→ out
i2──→┤0│    │ │
i3──→┤1├──→┤1│
     └─┘    └─┘
  s0          s1
```

3. Blocking vs Nonblocking — 핵심!

실행 순서 차이

```
// Blocking: 순서대로, 즉시 할당
initial begin
  x = #5 1'b0;   // time 5
  y = #3 1'b1;   // time 8 (5+3, blocked)
  z = #6 1'b0;   // time 14 (8+6)
end

// Nonblocking: RHS 동시 평가
initial begin
  x <= #5 1'b0;  // time 5  (모두 time 0에서 동시 시작!)
  y <= #3 1'b1;  // time 3
  z <= #6 1'b0;  // time 6
end
```

Shift Register — 결정적 차이

```
// Blocking (잘못! FF 1개만 합성)
always @(posedge clk) begin
  qout[0] = sin;       // 즉시 변경
  qout[1] = qout[0];   // sin의 새 값! (=sin)
  qout[2] = qout[1];   // 역시 sin!
  qout[3] = qout[2];   // 모두 sin!
end
```

합성 결과 (잘못됨):

```
sin ──→┤DFF├──→ qout[0]=qout[1]=qout[2]=qout[3]
       └───┘

clk     (FF 1개만!)
```

```
// Nonblocking (올바름! FF 4개 체인)
always @(posedge clk) begin
  qout[0] <= sin;       // 모든 RHS는 이전 값
  qout[1] <= qout[0];   // qout[0]_old
  qout[2] <= qout[1];   // qout[1]_old
  qout[3] <= qout[2];   // qout[2]_old
end
// 더 좋은 표현: qout <= {qout[2:0], sin};
```

합성 결과 (올바름):

```
sin──→┤DFF├──→qout[0]──→┤DFF├──→qout[1]──→┤DFF├──→qout[2]──→┤DFF├──→(
      └───┘              └───┘              └───┘              └───┘
                          clk (공유)
```

Race Condition 방지

```
// Blocking → race! (순서 의존)      // Nonblocking → 올바른 swap
always @(posedge clock) x = y;       always @(posedge clock) x <= y;
always @(posedge clock) y = x;       always @(posedge clock) y <= x;
```

Nonblocking 3단계: (1) 모든 RHS 읽기 (2) 수식 평가→임시 저장 (3) 임시 →LHS 할당.

count/finish 예제

count=1, finish=0에서 posedge clk:

```
// Blocking → finish=1              // Nonblocking → finish=0
always @(posedge clk) begin          always @(posedge clk) begin
  count = count-1;    // count=0      count <= count-1;    // RHS:
1-1=0
  if(count==0)                         if(count==0)         // count
는 아직 1!
    finish = 1;       // true!           finish <= 1;       //
false!
end                                  end
```

4. 5가지 가이드라인 (필수 암기!)

```
1. 순차 논리: always @(posedge clk) + nonblocking (<=)
2. 간단한 조합 논리: assign (dataflow)
3. 복잡한 조합 논리: always @(*) + blocking (=)
4. 같은 신호를 여러 always/assign에서 할당하지 마라
5. blocking과 nonblocking을 같은 블록에서 섞지 마라
```

5. Counter with 비동기 리셋

```
module counter(input clock, clear, output reg [3:0] qout);
  always @(negedge clock or posedge clear) begin
    if (clear) qout <= 4'd0;       // 비동기 리셋 우선
    else       qout <= qout + 1;   // negedge에서 카운트
  end
endmodule
```

합성 회로도:

```
qout[3:0]──→┤ +1 ├──→┤ 0 |
            └────┘   |MUX├──→┤D    Q├──→ qout[3:0]
4'd0 ──────────────→┤ 1 |   |DFF  |
                     └───┘   |  R  |
                clear──→ sel  └────┘
                              negedge clock
                (clear=1이면 비동기 즉시 리셋)
```

6. case/casex/casez

```
module mux_4x1_case(input I0,I1,I2,I3, input [1:0] S, output reg Y);
  always @(*)
    case (S)
      2'b00: Y=I0; 2'b01: Y=I1; 2'b10: Y=I2; 2'b11: Y=I3;
    endcase
endmodule
```

```
합성 회로도 (4:1 Mux):

I0──→┤ ├ I1──→┤ 4:1 ├──→ Y I2──→┤ MUX │ I3──→┤ │ └──────┘
S[1:0]──→┤sel│
```

```
// Trailing zero counter (casex)
module trailing_zero_4b(input [3:0] data, output reg [2:0] out);
  always @(data)
    casex (data)
      4'bxxx1: out=0;  4'bxx10: out=1;
      4'bx100: out=2;  4'b1000: out=3;
      4'b0000: out=4;  default: out=3'b111;
    endcase
endmodule
```

합성 회로도 (priority encoder):

```
data[0]──→┐
data[1]──→┤ Priority   ├──→ out[2:0]
data[2]──→┤ Encoder    │
data[3]──→┤ (casex에서 │
          │  x=don't   │
          │  care)     │
          └────────────┘
```

7. Timing Control

Inter-assignment: #10 a=b; → 10단위 후 문장 전체 실행. Intra-assignment: a=#10 b; → RHS 즉시 평가, 10단위 후 할당.

posedge: 0→1,0→x,0→z,x→1,z→1. negedge: 1→0,1→x,1→z,x→0,z→0.

Named event: event declared → trigger(->) → recognized(@). Level-sensitive: wait(condition).

8. Loop Constructs

Zero counter (for)

```
module zero_count_for(input [7:0] data, output reg [3:0] out);
  integer i;
  always @(data) begin
    out = 0;
    for (i=0; i<=7; i=i+1)
      if (data[i]==0) out = out + 1;
  end
endmodule
```

합성 회로도 (for는 unroll됨):

```
data[0]─→┤==0?├─→┐
data[1]─→┤==0?├─→┤
data[2]─→┤==0?├─→┤ Population
data[3]─→┤==0?├─→┤ Count    ──→ out[3:0]
data[4]─→┤==0?├─→┤ (zero의
data[5]─→┤==0?├─→┤  개수)
data[6]─→┤==0?├─→┤
data[7]─→┤==0?├─→┘
```

8-bit Left Shift Register (for + nonblocking)

```
module lshift_reg(input clk, rstn, input [7:0] load_val,
                  input load_en, output reg [7:0] op);
  integer i;
  always @(posedge clk) begin
    if (!rstn) op <= 0;
    else if (load_en) op <= load_val;
    else begin
      for (i=0; i<7; i=i+1)
        op[i+1] <= op[i];   // 각 비트가 다음으로 시프트
      op[0] <= op[7];       // circular
    end
  end
endmodule
```

합성 회로도:

```
op[7]──→op[0]──→op[1]──→op[2]──→...──→op[7]
       (circular left shift, 8개 DFF 체인)

rstn=0: 전체 0으로 리셋
load_en=1: load_val 로드
else: 매 posedge마다 좌로 1비트 순환 시프트
```

repeat: 고정 횟수 반복. forever: 무한 반복, $finish/disable로 종료.

```
// forever로 비대칭 클럭 생성
initial begin
  clock <= 0;
  forever begin #10 clock<=1; #5 clock<=0; end
end
```

```
clock: _____|__________|_____|__________|_____
        0    5         15    20         30
        (HIGH=10ns, LOW=5ns, 주기=15ns)
```